

Permissions, safety, and guardrails

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 12 of 12 · English notes

The scary story was simple: Claude might emit `rm -rf /` (or `/root`) and Bash might obey. Built-in tools are powerful; **permission gates** and **hooks** are how an enterprise agent stays on the right side of the disk. The trainer counted **six permission gates**, **five permission modes**, and a critical “do not loop forever” control — then showed combinations in a `run_agent` wrapper.

Six gates (classroom list)

Gate	What it does	Typical use
Pre-tool-use hooks	Inspect the impending call; stop/deny with a reason	Block dangerous bash patterns before they run
Disallowed tools	Hard deny list — never execute, even if the model asks	No Write/Edit on a read-only reviewer agent
Allowed tools	Allow list of built-in, custom, or MCP tools	Only Read, Glob, Grep for observe-only
Permission modes	Whether to prompt a human, auto-approve, or deny by default	See the five modes below
<code>can_use_tool</code>	Callback when skip/interactive collection is needed	Ask the user; gather extra input
No-callback / non-interactive	Sessions without a human; avoid infinite tool ping-pong	Logged-in batch jobs

Disallowed wins. If Bash is allowed but `rm *` is also disallowed (or a hook denies it), the deny path has priority. Allowed tools mean “eligible,” not “already approved for every argument.”

CHECK LIVE ENUM NAMES

Slides listed default, acceptEdits, bypassPermissions, don’t ask, plan, and an “auto” the linter also knew. Match strings to current Agent SDK / Claude Code permission docs, not to a screenshot from 27 August 2026.

Five modes he wanted you to contrast

- **Default** — standard behaviour; prompt on dangerous operations; unmatched tools can hit `can_use_tool`. Most general interactive state.
- **Don’t ask** — no permission UI; anything not pre-approved on the allow list is *denied*, but the agent *continues* with later steps (write fails, a later Read may still run).
- **Accept edits** — auto-approve filesystem edits that are already on the allow list.
- **Bypass permissions** — treat listed tools as approved at the gate. He called this dangerous; only with a tight allow list.

- **Plan** — read-only planning; no file or system edits. Use while surveying; switch to accept-edits when you intend to write.

Spectrum examples

Observe only: allow Read, Glob, Grep; mode don't-ask. Even if the model emits Write, it cannot land. **Read-write on a known set:** allow Read/Write/Edit (and maybe Bash); don't-ask or accept-edits. MCP tools not on the list stay off in don't-ask. **Plan then act:** plan-mode agent to draft; second agent with accept-edits to apply.

What the notebook showed

Prompt: read `auth.py` and save a one-sentence summary to a demo folder. With Write *not* allowed and don't-ask, the result was: write attempted, permission denied, files unchanged, sometimes an offer to enable Write. The same prompt with Write allowed created `summary_b.txt`. A later run showed don't-ask skipping a blocked Write/Bash and still executing a following Read.

Surgical blocking with pre-tool hooks

Allowing Bash wholesale is blunt. Hooks (pre-tool-use, post-tool-use, failure, prompt submit, stop, subagent start/stop, pre-compact, permission request, notification) let you pattern-match. The repo listed dangerous bash patterns: if the tool is Bash and the command matches, return deny plus reason (`continue = false` in the matcher). That stopped `rm -rf` on a blocking-demo folder even though Bash was allowed.

Second pattern: **redirect writes** away from a protected path into a temp directory so you do not throw away billed tokens. LLM output is expensive; if Write is denied on `/protected`, hook-rewrite to a safe folder and log it.

Guardrails as input/output validation

Permissions stop tools. **Guardrails** in this hour were mostly developer-owned checks around the prompt and the model/tool output: block “ignore previous instructions” / “reveal the system prompt,” cap prompt length, scan for secrets, redact. Apply *before query* (save tokens) and *after* (do not log API keys the model echoed). Vijay agreed these are general engineering, not a hidden Claude token-limiter. He pointed at open-source lists (Rulebricks, Claude Code guardrails-style repos) rather than claiming a single official Python import. Official evaluation guidance lives on Anthropic’s strengthen-guardrails page.

PII example that is not “encrypt at the gateway”

Rishabh asked whether HIPAA/PII meant Protegrity-style encryption into the gateway. Vijay’s agent answer: a customer-DB agent that builds a report and sends email *must* see a real address to send, but logs, MCP hops, and third-party tools must not leak that address. Permissions, hooks, and output scanners are the Claude-app layer; they do not replace your organisation’s encryption architecture.

Official documentation

- **Strengthen guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Anthropic's evaluation and guardrail guidance.

- **Usage policy**

<https://www.anthropic.com/legal/usage-policy>

What you may not build, regardless of hooks.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Permission modes in the product you already used in week 2–3.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Custom tools still pass through the same gates if the SDK wraps them.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.